

Weighted Interval Scheduling

You are given a set of n requests (intervals), labeled $1, 2, \dots, n$. For each interval i , you are given:

- s_i : A start time
- f_i : A finish time
- v_i : A value (or weight)

Compatibility: Two intervals are compatible (or disjoint) if they do not overlap in time.

Goal: Select a subset S of mutually compatible intervals to maximize the sum of values of the selected intervals (i.e., maximize $\sum_{i \in S} v_i$)

Crucial Setup (Sorting): Before solving, we always assume the requests are sorted in order of non-decreasing finish time. This is the natural left-to-right order we will use to process them.

$$f_1 \leq f_2 \leq f_3 \leq \dots \leq f_n$$

We say a request i comes before request j if $i < j$.

The $P(j)$ Function

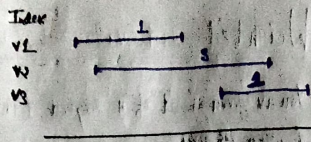
To build an efficient algorithm, we need a way to easily identify which previous intervals ~~are~~ are compatible with the current one. We define a helper function, $P(j)$.

Definition of $P(j)$: For any interval j , $p(j)$ is defined as the largest index $i < j$ such that intervals i and j are disjoint.

- In plain English: $p(j)$ is the leftmost interval that ends before j begins.
- If no such interval exists (i.e., all intervals before j overlap with it), we define $p(j) = 0$.

Why Greedy Algorithm Fails

In the unweighted version of this problem (where all $v_i = 1$), a greedy algorithm that always picks the interval that ends earliest provides the optimal solution. Let's see why this fails when weights are introduced.



Iterative Greedy Attempt (Earliest Finish Time):

1. The algorithm picks interval 1 because it ends first. (Current Value = 1)
2. It looks at interval 2. It is overlapping, so it is rejected.
3. It looks at interval 3. Interval 3 is compatible with interval 1. (Current value = 2)

>> Greedy Solution = 2

The Optimal Solution:

- If we simply pick interval 2, it conflicts with 1 and 3 but its value is 3.

>> Optimal Solution = 3

The greedy approach is not optimal. We need a recursive/dynamic programming approach.

The Recurrence Relation

To solve this optimally, we must evaluate a fundamental dichotomy for the very last interval n .

Definitions:

- let O_j be the optimal set of compatible intervals from the set $\{1, 2, \dots, j\}$.
- let $OPT(j)$ denote the total maximum weight (value) of the optimal solution O_j .

The Recurrence Equation: For any interval $j \geq 1$, the optimal value $OPT(j)$ can be defined recursively as:

$$OPT(j) = \max(v_j + OPT(p(j)), OPT(j-1))$$

Base Case: $OPT(0) = 0$ (the optimum over an empty set of intervals).

Inclusion Condition: How do we know if interval j is actually part of the optimal solution O_j ? Interval j belongs to the optimal solution on the set $\{1, 2, \dots, j\}$ if and only if:

$$v_j + OPT(p(j)) \geq OPT(j-1)$$

The Naive Recursive Algorithm & Its Correctness

Using the recurrence relation directly, we can write a simple recursive algorithm. We assume the requests are already sorted by finish time and $p(j)$ has precomputed for all j .

Compute-Opt(j)

If $j=0$ then

Return 0

Else

Return $\max(v_j + \text{Compute-opt}(p(j)), \text{Compute-opt}(j-1))$

Endif

Theorem $\rightarrow$ Compute-Opt(j) correctly computes $OPT(j)$ for each $j = 1, 2, \dots, n$.

Proof $\rightarrow$ By induction on j .

- Base Case: By definition, $OPT(0) = 0$. The algorithm explicitly returns 0 when $j=0$, so the base case holds.

- Inductive Step: Take some $j > 0$. Assume, by way of the induction hypothesis, that

Compute-Opt(i) correctly computes $OPT(i)$ for all $i < j$.

- Since $p(j) < j$ and $(j-1) < j$, our induction hypothesis guarantees that:
 1. $\text{Compute-opt}(p(j)) = OPT(p(j))$
 2. $\text{Compute-opt}(j-1) = OPT(j-1)$

- Therefore, looking at the Else branch of our algorithm:

$$\text{Compute-opt}(j) = \max(v_j + \text{Compute-opt}(p(j)), \text{Compute-opt}(j-1))$$

$$\text{Compute-opt}(j) = \max(v_j + OPT(p(j)), OPT(j-1))$$

- Based on our core recurrence equation, the right side of this equation is exactly $OPT(j)$. Thus, $\text{Compute-opt}(j) = OPT(j)$, completing the proof.

The Problem with Naive Recursion: Exponential Time

While Compute-Opt is mathematically correct, it takes exponential time. This happens because the algorithm blindly re-evaluates the same subproblems over and over.

For example, Compute-Opt(6) might branch into calculating Compute-Opt(3) and Compute-Opt(5). But Compute-Opt(5) branches into another independent call to Compute-Opt(3). This redundancy explodes exponentially. If an instance is structured such that $p[j] = j-2$, the recurrence mirrors the Fibonacci sequence ($F_n = F_{n-1} + F_{n-2}$), proving an $O(\phi^n)$ exponential time complexity.

Memorization (Top-Down Dynamic Programming)

A fundamental observation is that Compute-Opt is really only solving $n+1$ different subproblems: Compute-Opt(0) through Compute-Opt(n).

Memorization: We can store the value of Compute-Opt in a globally accessible array M the first time we compute it, and then simply use this precomputed value for all future recursive calls.

$M[0 \dots n] \leftarrow$ global memo.

$M \leftarrow$ Compute-Opt(j)

If $j=0$ then
return 0

Else if $M[j]$ is not empty then
return $M[j]$

Else

define: $M[j] = \max(v_j + M[\text{Compute-Opt}(p[j])], M[\text{Compute-Opt}(j-1)])$

return $M[j]$

End if.

This reduces the time complexity to $O(n)$ because we fill exactly n array slots in calculated exactly once taking $O(1)$ time.

Iteration (Bottom-Up DP)

While memorization solves the time complexity issue, it still relies on recursion. We can take this a step further. If we know we just need to fill an array M of size $n+1$, and every entry $M[j]$ only depends on strictly smaller entries ($M[p[j]]$ and $M[j-1]$), we can simply fill the array iteratively from left to right. This is the standard dynamic programming template.

Iterative-Compute-Opt

$M[0] = 0$

For $j=1, 2, \dots, n$

$M[j] = \max(v_j + M[p[j]], M[j-1])$

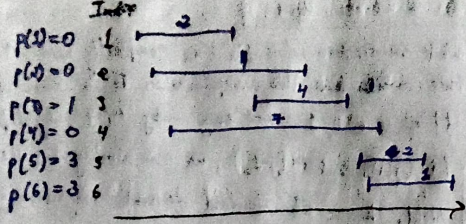
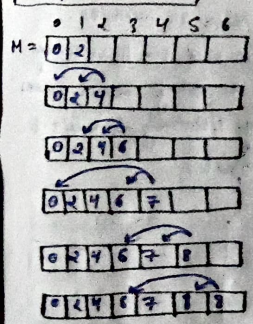
End-for

Theorem $\rightarrow$ Iterative-Compute-Opt correctly computes OPT(j) for each $j=1 \dots n$ and runs in $O(n)$ time.

Proof $\rightarrow$ Correctness (by induction)

- Base Case:** The algorithm explicitly sets $M[0] = 0$, which is OPT(0).
- Inductive Step:** Assume $M[i]$ correctly stores OPT(i) for all $i < j$. When the loop reaches j , it computes $M[j] = \max(v_j + M[p[j]], M[j-1])$. Because $p[j] < j$ and $j-1 < j$, we know $M[p[j]] = \text{OPT}(p[j])$ and $M[j-1] = \text{OPT}(j-1)$. Thus, the formula evaluates to exactly the recurrence relation (6.1), proving $M[j] = \text{OPT}(j)$.
- Running Time:** The algorithm explicitly runs a For loop exactly n times. Inside the loop, it performs a single array lookup, an addition, and a maximum comparison - all of which take $O(1)$ constant time. Therefore, the total running time is clearly $O(n)$.

Example Iteration



Instance Data:

$v = \{4, 4, 4, 7, 2, 1\}$ (The values of the 6 intervals)
 $p = \{0, 0, 1, 0, 3, 3\}$ (The precomputed compatible predecessors)

The Execution Iterations:

• Initialization: $M[0] = 0$. Array v is: $[0, -, -, -, -, -, -]$

j=1 Compare: Include 1 ($v_1 + M[p(1)] = 2 + M[0] = 2$) vs. Exclude 1 ($M[0] = 0$)

$$M[1] = \max(2, 0) = 2$$

Array v is: $[0, 2, -, -, -, -, -]$

j=2 Compare: Include 2 ($v_2 + M[p(2)] = 4 + M[0] = 4 + 0 = 4$) vs. Exclude 2 ($M[1] = 2$)

$$M[2] = \max(4, 2) = 4$$

Array v is: $[0, 2, 4, -, -, -, -]$

j=3 Compare: Include 3 ($v_3 + M[p(3)] = 4 + M[1] = 4 + 2 = 6$) vs. Exclude 3 ($M[2] = 4$)

$$M[3] = \max(6, 4) = 6$$

Array v is: $[0, 2, 4, 6, -, -, -]$

j=4 Compare: Include 4 ($v_4 + M[p(4)] = 7 + M[0] = 7 + 0 = 7$) vs. Exclude 4 ($M[3] = 6$)

$$M[4] = \max(7, 6) = 7$$

Array v is: $[0, 2, 4, 6, 7, -, -]$

j=5 Compare: Include 5 ($v_5 + M[p(5)] = 2 + M[3] = 2 + 6 = 8$) vs. Exclude 5 ($M[4] = 7$)

$$M[5] = \max(8, 7) = 8$$

Array v is: $[0, 2, 4, 6, 7, 8, -]$

j=6 Compare: Include 6 ($v_6 + M[p(6)] = 1 + M[5] = 1 + 8 = 9$) vs. Exclude 6 ($M[5] = 8$)

$$M[6] = \max(9, 8) = 9$$

Array v is: $[0, 2, 4, 6, 7, 8, 9]$

Maximum possible value found!

Recovering the Actual Intervals

Now that M is filled, we want the actual set of optimal intervals. We avoid a naive $O(n^2)$ array-copying approach by tracing backwards through our completed array M .

Using our Inclusion Condition, we know interval j is in the optimal set if and only if:

$$v_j + M[p(j)] \geq M[j-1]$$

Find-Solution(j)

If $j=0$ then
output nothing

Else

If $v_j + M[p(j)] \geq M[j-1]$ then
output j together with the result of Find-Solution($p(j)$)

Else

output the result of Find-Solution($j-1$)

Endif

Endif

Theorem $\rightarrow$ Given the array M , find-Solution returns an optimal solution in $O(n)$ time.

proof $\rightarrow$ find-Solution(j) calls itself recursively only on strictly smaller values (either $p[j]$ or $j-1$). Therefore, it can make a maximum of j recursive calls. The work done inside each call (a single array lookup, an addition, and a comparison) takes constant $O(1)$ time, yielding a total time of $O(n)$.

General Principles of Dynamic Programming

The solution to weighted Interval Scheduling serves as a general template for designing DP algorithms. To utilize DP, the problem must be broken down into a collection of subproblems that satisfy three basic properties:

1. **Polynomial Subproblems:** There are only a polynomial number of subproblems. (e.g., we only had $n+1$ subproblems for $j = 0, 1, \dots, n$).
2. **Easy Reconstruction:** The solution to the original problem can be easily computed from the solutions to the subproblems. (e.g., the original problem is simply $M[n]$).
3. **Natural Ordering & Recurrence:** There is a natural ordering on subproblems, from "smallest" to "largest", together with an easy-to-compute recurrence that allows one to determine the solutions to smaller subproblems. (e.g., $M[j] = \max(j, M[p[j]])$, $M[j-1]$).

Subset Sum and Knapsack Problems

These problems revolve around scheduling jobs with durations and a strict deadline (or packing items with weights into a bounded container).

The Subset Sum Problem

Setup: You are given a set of n items, labeled $1, 2, \dots, n$.

- Each item i has a non-negative integer weight (or time duration), denoted by w_i .
- You have a maximum capacity (or time cut-off bound), denoted by W (also assumed to be an integer).

Goal: Select a subset S of the items to maximize the total weight $\sum_{i \in S} w_i$, subject to the strict restriction that the total weight cannot be exceed W .

$$\sum_{i \in S} w_i \leq W$$

The Knapsack Problem (Generalization)

The Knapsack Problem is a more complex generalization of Subset Sum.

- Each item i now has both a weight w_i AND a distinct value v_i .

Goal: Maximize the total value $\sum_{i \in S} v_i$, subject to the same weight capacity restriction

$$\sum_{i \in S} w_i \leq W.$$

Why Greedy Algorithm Fails

Because Subset Sum resembles earlier scheduling problems, it is tempting to use a simple greedy algorithm. Let's look at two natural greedy strategies and trace exactly why they fail to find the optimal solution.

Attempt 1: Greedy by decreasing weight

Strategy: Sort items from the heaviest to the lightest. Pick the heaviest items first, as long as they fit in the remaining capacity W .

Let the total capacity $W = 10$.

We have 3 items with weights: $\{6, 5, 3\}$. $\leftarrow$ already sorted

Iterative Greedy Attempt:

Item 1 (Weight 6): Fits in capacity 10. (Remaining 4)

Item 2 (Weight 5): Does not fit in remaining capacity.

Item 3 (Weight 5): Does not fit in remaining capacity.

Greedy Solution = 6

The Optimal Solution:

If we skip the heaviest item, we can pack the two items of weight 5.

Optimal Solution = 10.

Attempt 2: Greedy by Increasing Weight

Strategy: Sort items from lightest to ~~lightest~~ heaviest. Pick the lightest item first to try and pack as many individual item as possible.

Let the total capacity $W = 10$

We have 3 items with weight: $\{1, 5, 5\}$. $\leftarrow$ Already-Sorted

Iterative Greedy Attempt:

Item 1 (Weight 1): Fits in $W(10)$ (Remaining 9)

Item 2 (Weight 5): Fits in $W(9)$ (Remaining 4)

Item 3 (Weight 5): Cannot fit.

Greedy Solution = 6

The Optimal Solution:

If we skip the lightest item, we can fit the two heavier items.

Optimal Solution = 10

The Need to "Add a Variable" (A False Start)

To solve this using DP, we must follow the standard principle: define a set of subproblems that are ~~easy~~ easy to link together.

In previous problems (like weighted interval scheduling), one subproblem just tracked the number of items: $OPT(i)$ was the optimal solution using the first i items.

Why $OPT(i)$ is a False Start here:

If we only track the index i , we know that if item $n \in O$, then $OPT(n) = OPT(n-1)$. However, if $n \notin O$, we have a problem. Accepting request n does not explicitly reject a specific previous item (like overlapping intervals did). Indeed, it means we have less available capacity left for the ~~the~~ remaining items. Knowing the optimal value for the first $n-1$ items in a vacuum doesn't help us, because we don't know if they will fit in the new, smaller remaining capacity.

A Better Solution: We must create a richer collection of subproblem by adding a new variable. To find $OPT(n)$, we need to know the best solution using a subset of the first $n-1$ items with a total allowed weight of ~~exactly~~ exactly $W - w_i$. We must track both the items we are considering (1) AND the available capacity.

The 2D Recurrence Relation

We assume W is an integer, and all items have integer weights. This allows us to create discrete subproblems for every capacity from 0 to W .

Let $OPT(i, w)$ denote the maximum weight we can pack using a subset of the first i items $\{1, \dots, i\}$, such that their total weight is $\leq w$.

Theorem $\rightarrow$ For any item $i \geq 1$ and any capacity $w \geq 0$, the optimal value

$OPT(i, w)$ is computed as follows: Base case $\Rightarrow OPT(0, w) = 0$

1. If $w < w_i$: The current item is strictly heavier than the available capacity.

$$\Rightarrow OPT(i, w) = OPT(i-1, w)$$

2. Otherwise (if $w \geq w_i$): The ~~current~~ current item fits in the available capacity.

$$\Rightarrow OPT(i, w) = \max(OPT(i-1, w), w_i + OPT(i-1, w-w_i))$$

proof $\rightarrow$ We want to find the optimal packing for the first i items with a weight limit of w . Let this optimal subset be O . Just like in previous scheduling problems, we face a fundamental dichotomy for item i : either item $i \in O$ or item $i \notin O$.

Case 1: $i \notin O$

- If item i is not in the optimal solution, then the optimal weight must be formed entirely from the first $i-1$ items, using the full available capacity w .
- The maximum weight is exactly $OPT(i-1, w)$.
- (If $w < w_i$, this is the only valid case, as the item physically cannot fit. This proves the first part of the theorem.)

Case 2: $i \in O$

- This case is only possible if $w_i \leq w$.
- If we include item i , we immediately consume w_i units of our capacity. (Remaining $\rightarrow w - w_i$)
- In optimal way to pack the first $i-1$ items into a capacity of $w - w_i$, is $OPT(i-1, w - w_i)$.
- If $i \in O$, the maximum weight is $w_i + OPT(i-1, w - w_i)$.

Since the optimal subset O must either include item i or exclude it, the overall weight is simply the maximum of these 2 mutually exclusive choices. This paper yields the recurrence relation (6.8), proving the theorem.

Iterative Algorithm

Using our 2D recurrence relation we can design a dynamic programming algorithm that builds up a table of all $OPT(i, w)$ values. Because w and w_i are integers, we can use a 2D array M where $M[i, w]$ represents $OPT(i, w)$.

Subset-Sum(n, w)

Array $M[0 \dots n, 0 \dots W]$

4 Base Case: 0 items available means 0 weight packets.

Initialize $M[0, w] = 0$ for each $w = 0, 1, \dots, W$.

For $i = 1, 2, \dots, n$

For $w = 0, \dots, W$

If $w < w_i$ then

$M[i, w] = M[i-1, w]$

Else

$M[i, w] = \max(M[i-1, w], w_i + M[i-1, w - w_i])$

Endif

Endfor

Endfor

Return $M[n, W]$

Visualizing the 2D Table

To understand how the algorithm fills the array M , picture a grid where the rows represent the items being considered (from 0 to n), and the columns represent the available weight capacities (from 0 to W).

When calculating $M[i, w]$, the algorithm looks at exactly two previously computed cells:

1. The cell directly above it: $M[i-1, w]$ (excluding item i)
2. The cell in the row above, shifted left by w_i columns: $M[i-1, w - w_i]$ (including item i).

Example Iteration

Knapsack size $W = 6$, item $w_1 = 2$, $w_2 = 2$, $w_3 = 3$

3							
2							
1							
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

Initial values

3							
2							
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

for $i=1$

3							
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

for $i=2$

3	0	0	2	3	4	5	5
2	0	0	2	2	4	4	4
1	0	0	2	2	2	2	2
0	0	0	0	0	0	0	0
	0	1	2	3	4	5	6

for $i=3$

$$i=3 (w_3=3)$$

For $w=0$: Item too heavy ($w < 3$). Inherit from above: $M[3, w] = M[2, w]$
The values are 0, 0, 2.

For $w=3$:
 $\max(M[2, 3], 3 + M[2, 0]) = \max(2, 3+0) = 3$.

For $w=4$:
 $\max(M[2, 4], 3 + M[2, 1]) = \max(4, 3+0) = 4$.

For $w=5$:
 $\max(M[2, 5], 3 + M[2, 2]) = \max(4, 3+2) = 5$.

For $w=6$:
 $\max(M[2, 6], 3 + M[2, 3]) = \max(4, 3+2) = 5$.

>> Row 3 is: 0 0 2 3 4 5 5

The final answer $\rightarrow M[3, 6] = 5$.

~~Theorem~~ $\rightarrow$ The Subset Sum (n, W) algorithm correctly computes the optimal value of the problem.

Theorem $\rightarrow$ The Subset Sum (n, W) algorithm correctly computes the optimal value of the problem and runs in $O(nW)$ time.

Proof $\rightarrow$ The table M has $n+1$ rows and $W+1$ columns, giving roughly nW entries. Computing the value of each entry requires checking the $w < w_i$ condition, performing an addition, and taking the maximum of two previously calculated numbers. All of this takes $O(1)$ time. Therefore, filling the entire table takes $O(nW)$ time.

Concept $\rightarrow$ Unlike Weighted Interval Scheduling (which ran in strictly polynomial $O(n)$ time), Subset Sum runs in $O(nW)$ time. The running time depends not just on the number of inputs (n), but on the magnitude of the largest integer involved in the problem (W).

$$i=1 (w_1=2)$$

For $w=0, 1$: The item is too heavy ($w < 2$). Inherit from above: $M[1, w] = M[0, w]$.

For $w \geq 2$: The item fits. $M[1, w] = \max(0, 2 + M[0, w-2]) = \max(0, 2) = 2$.

>> Row 1 is: 0 0 2 2 2 2 2

$$i=2 (w_2=2)$$

For $w=0, 1$: Item too heavy. Inherit from above: $M[2, w] = 0$.

For $w=2, 3$: The item fits. Compare excluding vs. including.
 $M[2, w] = \max(M[1, w], 2 + M[1, w-2])$.

• $w=2 \rightarrow$ this is $\max(0, 2+0) = 2$

• $w=3 \rightarrow$ this is $\max(2, 2+0) = 2$.

For $w=4, 5, 6$: The item fits, and taking it leaves enough room ($w-2 \geq 3$) to also have taken item 1. $M[2, w] = \max(M[1, w], 2 + M[1, w-2]) = \max(2, 2+2) = 4$.

• $w=4 \rightarrow \max(M[1, 4], 2 + M[1, 2]) = \max(2, 2+2) = 4$

>> Row 2 is: 0 0 2 2 4 4 4

If w is a massive number, the table becomes enormous. Algorithms whose running time depends on the magnitude of the numeric inputs are called pseudo-polynomials. They are highly efficient for reasonably small numbers but become impractical as the numbers grow exponentially large.

Recovering the Actual Solution

After computing the optimal value, we must trace back through the array to find which specific items belong to the optimal subset S .

Theorem $\rightarrow$ Given a table M of the optimal values of the subproblems, the optimal set S can be found in $O(n)$ time.

logic $\rightarrow$ Start at the bottom right corner of the table: $M[n, w]$. For any item i and current capacity w :

1. If $M[i, w] == M[i-1, w]$, it means item i was not used to achieve the optimal weight. We simply move up one row to $M[i-1, w]$.
2. Otherwise (if $M[i, w] \neq M[i-1, w]$), then item i must be in the optimal solution. We output item i , subtract its weight from the capacity ($w = w - w_i$), and move to $M[i-1, w - w_i]$.

Because we only ever move up rows, we make exactly n steps.

The comparisons at each step take $O(1)$ time, yielding an overall recovery time of $O(n)$.

The Full Knapsack Problem

The Knapsack Problem builds directly on top of Subset Sum. We are no longer just trying to fill capacity; we want to maximize the distinct value (v_i) of the items packed.

Setup: Each item i has a non-negative weight w_i AND a distinct value v_i .

Goal: Maximize $\sum_{i \in S} v_i$ subject to $\sum_{i \in S} w_i \leq W$.

Let $OPT(i, w)$ denote the maximum value we can pack using a subset of items $\{1, \dots, i\}$ with a maximum capacity w .

Theorem $\rightarrow$ For any item $i \geq 1$ and capacity $w \geq 0$:

1. If $w < w_i$: $OPT(i, w) = OPT(i-1, w)$
2. Otherwise: $OPT(i, w) = \max(OPT(i-1, w), v_i + OPT(i-1, w - w_i))$

proof $\rightarrow$ Consider an optimal solution O . Just like before:

- If $n \notin O$: We ignore item n . The optimal value is just the optimal value of the remaining $n-1$ items with the full capacity W .
 $\rightarrow OPT(n-1, W)$
- If $n \in O$: We include item n . We immediately gain its value v_n (instead of just its weight). However, it still consumes w_n units of physical capacity. We must optimally fill the remaining $W - w_n$ capacity using the first $n-1$ items.
 $\rightarrow v_n + OPT(n-1, W - w_n)$

The optimal choice is simply the maximum of those two options.

Algorithm & Complexity: By replacing w_i addition with a v_i addition in the Subset Sum problem, we solve the full Knapsack problem. Because the structure of the 2D table and the subproblem dependencies are identical, the Knapsack Problem can also be solved in $O(nw)$ pseudo-polynomial time.